Professional development for information security specialists

LECTURE NOTES

course:
“Methods and Principles of Designing Modern Secure Systems. Using Artificial Intelligence Methods for Data Analysis in Information Security Systems”

LECTURE 1. DESIGNING SECURE SYSTEMS

Assets, trust boundaries, adversary, threats, security requirements

(2 academic hours)

2026

Contents

1 Introduction 4

2 Security properties and design principles 5

2.1 Security properties 5

2.2 Design principles 5

2.3 Defense in depth 6

2.4 Principles broken in real incidents 6

3 Assets and attack surface 8

3.1 Assets 8

3.2 Attack surface 8

3.3 Reducing the attack surface 9

4 Trust boundaries and the data flow diagram 10

4.1 Trust boundaries 10

4.2 Data flow diagram 10

5 Adversary model and threats 12

5.1 Adversary model 12

5.2 Attack scenarios 12

5.3 STRIDE threat types 13

5.4 Risk assessment 14

6 Zero trust architecture 16

6.1 Why zero trust 16

6.2 Tenets of zero trust 16

6.3 Components 17

6.4 Zero trust in the shop 17

7 Security requirements 18

7.1 Sources of requirements 18

7.2 OWASP ASVS 18

7.3 How to write a requirement 18
7.4 Threat modeling process 19
8 Conclusion 20
References 21
9 Self-check questions 23

1 Introduction

Sl. 1, 2

This lecture is about designing secure systems. The idea is simple. Think about security in the architecture first, before you write code. Write the security requirements before the work starts, and check them at every stage. Most serious problems come from architecture decisions, not from single lines of code [1]. A wrong architecture cannot be fixed with a small patch. The system must be changed.

A good example is the Equifax leak in 2017. Attackers used an old, unfixed bug in Apache Struts. They got the data of about 147 million people. The US Federal Trade Commission named poor patch control as one of the causes [2]. Keeping a list of systems and installing patches are design tasks, not coding tasks.

One example runs through the whole lecture: an online shop. Customers use a browser. An administrator manages the catalog and the orders. The web application keeps data in a database and sends payments to an external payment service. In each chapter we apply one new step to this shop. At the end we get one table of threats and controls.

The lecture is a chain of steps. The result of one step is the input of the next step (table 1).

Table 1 — Steps of secure system design

Chapter	Question	Result
1	What do we protect, and by which rules?	Security properties and design principles
2	What is valuable, and where can it be reached?	List of assets and attack surface
3	Where does the level of trust change?	Data flow diagram with trust boundaries
4	Who can attack, and how?	Adversary model, threats, risks
5	How do we check every access?	Zero trust architecture
6	How do we write controls so that we can test them?	Security requirements and threat table

2 Security properties and design principles

2.1 Security properties

Sl. 4

Every security requirement protects one or more properties of information. Three main properties form the “CIA triad”: confidentiality, integrity and availability. The ISO/IEC 27000 standard [3] defines them like this. **Confidentiality** means information is not given to people or processes that have no right to it. **Integrity** means information is accurate and complete. **Availability** means information is ready to use when an authorized subject asks for it. Table 2 shows typical attacks and controls.

Table 2 — The CIA triad and its violations

Property	Meaning	Example of violation	Typical controls
Confidentiality	No unauthorized person can read the data	Data leak, traffic interception	Encryption, access control, masking of personal data
Integrity	Data is accurate and complete	SQL injection, data change in transit	Message authentication codes, digital signatures, hash chains
Availability	Data can be used on request	Denial of service, ransomware	Redundancy, backups, rate limiting

Two more properties are about responsibility. **Non-repudiation** means a person cannot deny an action later. **Authenticity** means a subject or a resource is really what it claims to be [3]. We need them when we must prove who did what.

Sl. 5

One weakness can break several properties at once. In 2023 attackers used an SQL injection in MOVEit Transfer. They stole data from many organizations (confidentiality). They also added hidden accounts (integrity). Some organizations switched the service off (availability).

2.2 Design principles

Sl. 6

In 1975 Saltzer and Schroeder wrote down principles for protection mechanisms [4]. They are still the base of modern standards, for example NIST SP 800-160 [5]. Table 3 lists eight principles.

Table 3 — The eight principles of Saltzer and Schroeder

Principle	Meaning	Example
Economy of mechanism	The protection mechanism is small and simple, so it can be checked	One central login service, not five different ones
Fail-safe defaults	Access is denied unless it is allowed	If the permission check fails, the request is rejected
Complete mediation	Every access is checked again	A token is checked on every request
Open design	Security does not depend on a secret design. Only the key is secret	AES and TLS are public, the keys are secret
Separation of privilege	A critical action needs more than one person	Code is reviewed and approved before release
Least privilege	A subject gets only the rights it needs	The order service writes only to the orders table
Least common mechanism	Share as little as possible between users	Separate databases for different customers
Psychological acceptability	Protection is easy to use, or people will avoid it	A simple login with a second factor

2.3 Defense in depth

Sl. 7

In practice we add one more idea: **defense in depth**. NIST defines it as a strategy that uses people, technology and operations to build barriers on many layers [6]. In a technical system this means several independent layers. If one layer fails, the attacker does not get everything. Figure 1 shows the layers.

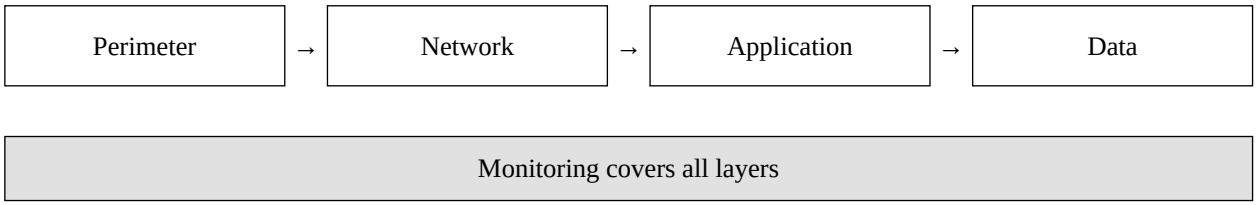


Figure 1 — Layers of defense in depth

Take a request to `/api/orders/{id}`. The perimeter blocks known attack patterns. The gateway checks the token and limits the request rate. The application checks that the order belongs to the user. The database accepts only parameterized queries. The log records the user, the order and the source address.

2.4 Principles broken in real incidents

Sl. 8

Big incidents often break one or more principles (table 4).

Table 4 — Examples of broken design principles

Incident	Broken principle	What happened
Capital One, 2019	Least privilege	A web firewall was misconfigured and a server-side request forgery (SSRF) worked. The attacker got a role with too much access to storage and copied data of about 100 million people [7]
SolarWinds, 2020	Separation of privilege	A backdoor was put into the build system. About 18,000 organizations received the update [8]
Log4Shell, 2021	Economy of mechanism	A logging library could look up external directories inside a logged string. A string from a user ran foreign code (CVE-2021-44228)

3 Assets and attack surface

3.1 Assets

An **asset** is anything that has value for the organization [3]. Examples are data, infrastructure, services and reputation. We build protection around assets, so we list them first. For each asset we note which property matters most. For secrets it is confidentiality. For services it is availability. For code and configuration it is integrity.

Sl. 10

In the training shop the assets are in table 5.

Table 5 — Assets of the training shop

Shop asset	Category	Most important property
Customer data: name, address, e-mail, order history	Personal data	Confidentiality
Payments and amounts	Financial data	Integrity, confidentiality
Key for the payment service	Secret	Confidentiality
Product catalog and prices	Data	Integrity
Website operation	Service	Availability

3.2 Attack surface

Sl. 11

The **attack surface** is the set of points where an attacker can try to get in, act on the system or take data out [6]. Such points are interfaces, forms, ports, services, accounts and also people. More points mean more chances for the attacker and more work for the defender. The points depend on who attacks. An **external** attacker has no authorized access. An **internal** attacker has some access, for example a user, an administrator or a contractor. An outsider looks for a way in. An insider goes beyond the rights they have. Table 6 compares them.

Table 6 — Attack surface of an external and an internal attacker

Attacker	Entry points	Ways in	How to reduce the surface
External	Internet-facing services and interfaces; e-mail; third-party libraries; employees	Bug in a public service; password guessing; injection; phishing; backdoor in a dependency	One gateway, web firewall, closing unused ports, signing artifacts, second factor
Internal	Workstations, admin panels, internal interfaces, databases, media	Abuse of rights; privilege escalation; moving between services; data theft; physical access	Least privilege, separation of privilege, segmentation, action log, disk encryption

In the shop, an outsider can reach the login page, the order form and the payment answers. An insider can reach the admin panel and the database.

3.3 Reducing the attack surface

Sl. 12

Michael Howard (Microsoft) described four steps (figure 2). First, list all entry points. Second, classify each one: who can reach it, what data it takes, what rights it needs. Third, remove what is not needed. Fourth, review it again, because new entry points appear with every release.

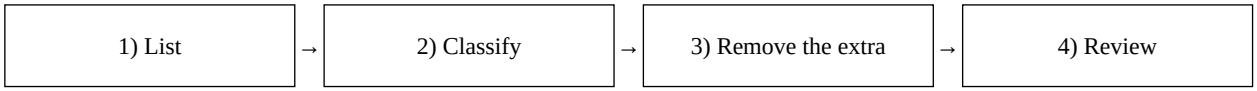


Figure 2 — Steps to reduce the attack surface

Good measures are: remove debug entry points, use one gateway instead of many entry points, and allow only the network connections that are needed.

4 Trust boundaries and the data flow diagram

4.1 Trust boundaries

Sl. 14

A **trust boundary** is a place where data moves from one trust zone to another [9]. Data from the internet is trusted less than data inside a protected network. So we do checks at the boundary: authentication, authorization and input validation. A simple rule: if you need authentication or encryption to cross between two zones, it is a trust boundary. Table 7 shows common boundaries.

Table 7 — Common trust boundaries

Boundary	Required checks	Typical attacks
Client → server	TLS, token check, input validation	Cross-site scripting, password guessing, injection
Service → service	Mutual authentication, permissions, parameterized queries	SSRF, SQL injection, privilege escalation
External system → ours	Key or OAuth, rate limit, format check	Abuse of the interface, data extraction
Build → production	Signed artifacts, admission policies	Supply chain attacks, image replacement
User → administrator	Second factor, full audit, allowed address list	Privilege escalation, account takeover
Cloud → local network	VPN or identity-aware access, firewall	Lateral movement, credential theft

4.2 Data flow diagram

The main tool of Microsoft threat modeling is the **data flow diagram** (DFD) [9]. It shows the parts of a system and the data that moves between them. It makes all trust boundaries visible. A DFD has five kinds of elements (table 8).

Sl. 15

Table 8 — Elements of a data flow diagram

Element	Symbol	Meaning
External entity	Rectangle	A user or an external system we do not control
Process	Circle	A component that handles data: a service, a function
Data store	Two parallel lines	Database, file system, cache
Data flow	Arrow	Data moving between elements
Trust boundary	Dashed line	Place where the level of trust changes

Sl. 16

Figure 3 shows the shop. A customer and an administrator use the web application from the internet. The application talks to the database in the internal network and to the external payment service. There are three trust boundaries and four flows. We will look for threats there.

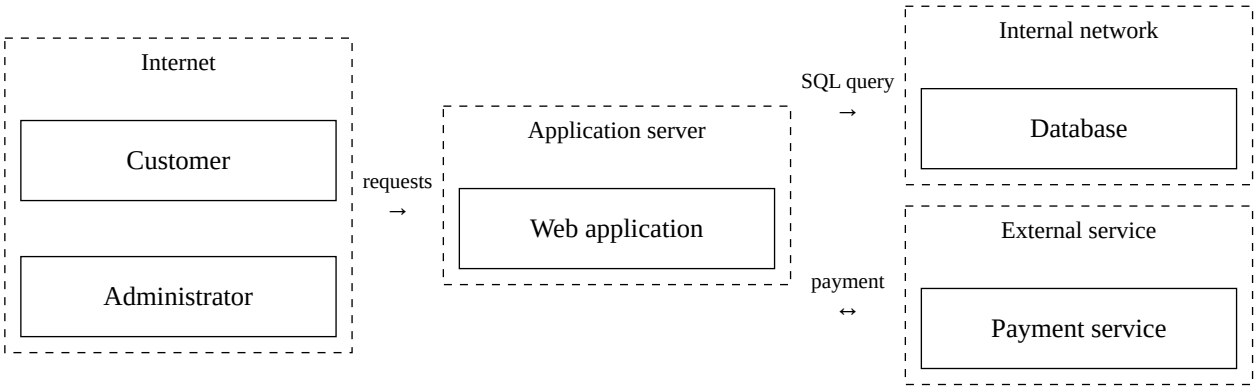


Figure 3 — Data flow diagram of the training shop (dashed lines are trust boundaries)

5 Adversary model and threats

5.1 Adversary model

Sl. 18

An **adversary** is a person, group, organization or state that does or plans to do harm [10]. An **adversary model** describes who can attack, why, and with what means. We need it to set priorities. A system that faces automatic scans is built in a different way from a system that faces a state group. Table 9 lists typical adversaries.

Table 9 — Typical adversaries

Adversary	Motive	Resources	Typical actions
Novice	Curiosity	Low	Ready-made exploits, scanners
Hacktivist	Ideology, protest	Low or medium	Overload, defacement, publishing data
Organized crime	Money	Medium or high	Ransomware, e-mail fraud, stolen passwords
State group	Spying, destruction	Very high	Unknown vulnerabilities, supply chain attacks
Insider	Revenge, profit	High (has access)	Data theft, sabotage, hidden access
Careless employee	No intent	Access without bad intent	Misconfiguration, leaked secrets

Sl. 19

Adversaries also differ in **capability**: knowledge, resources and motivation. The Common Criteria method has four levels. In Russian practice they are called N1–N4 (table 10). An outsider with a stolen employee account acts like an insider.

Table 10 — Adversary capability

Level	Capability	What the adversary can do
N1	Basic	Uses ready-made tools and well-known vulnerabilities
N2	Enhanced basic	Changes ready-made tools, combines several vulnerabilities
N3	Moderate	Finds vulnerabilities on their own and builds own tools
N4	High	Plants backdoors in software or hardware, has the resources of an organization or a state

5.2 Attack scenarios

Sl. 20

The **Cyber Kill Chain** of Lockheed Martin describes the steps of a targeted attack [11]. It has seven stages (figure 4). The model shows where we can notice and stop an attack. The earlier we stop it, the smaller the damage.

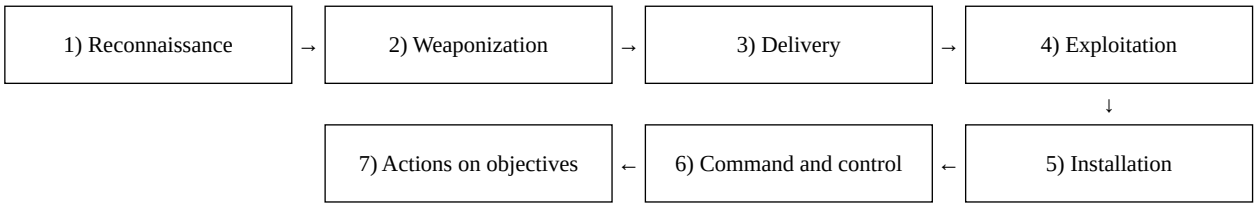


Figure 4 — Seven stages of a targeted attack

Sl. 21

MITRE ATT&CK gives more detail [12]. It splits the goals of an attacker (tactics) into techniques. These are linked to real groups and tools. Table 11 shows some examples.

Table 11 — Attack chain and ATT&CK tactics

Stage	ATT&CK tactic	Example technique	Detection
Reconnaissance	Reconnaissance (TA0043)	Active Scanning (T1595)	Web firewall logs
Delivery	Initial Access (TA0001)	Phishing (T1566)	Mail gateway, second-factor logs
Exploitation	Execution (TA0002)	Command and Scripting Interpreter (T1059)	Endpoint protection
Installation	Persistence (TA0003)	Scheduled Task/Job (T1053)	System audit
Command and control	Command and Control (TA0011)	Application Layer Protocol (T1071)	DNS and proxy analysis
Actions on objectives	Exfiltration (TA0010)	Exfiltration Over C2 Channel (T1041)	Data loss prevention

5.3 STRIDE threat types

Sl. 22

A **threat** is a possible cause of an unwanted incident that can harm a system or an organization [3]. **STRIDE** from Microsoft is a list of six threat types. The name is made from their first letters. Each type breaks one security property (table 12) [13].

Table 12 — STRIDE threat types

Letter	Threat	Property broken	Example
S	Spoofing	Authenticity	Using a stolen password or a fake token
T	Tampering	Integrity	Changing a payment amount in transit
R	Repudiation	Non-repudiation	An administrator denies deleting records
I	Information disclosure	Confidentiality	Database dump through a vulnerability
D	Denial of service	Availability	Flood of requests to the login page
E	Elevation of privilege	Authorization	A normal user runs an admin function

Sl. 23

We apply STRIDE to the elements of the DFD. Not every type fits every element (table 13). For example, elevation of privilege is possible only for a process, because only a process performs actions with rights.

Table 13 — Threat types for each DFD element (* log stores only)

Element	S	T	R	I	D	E
External entity	✓		✓			
Process	✓	✓	✓	✓	✓	✓
Data store		✓	✓*	✓	✓	
Data flow		✓		✓	✓	

Sl. 24

The result is a list of threats. Table 14 lists the threats for the shop. We keep these numbers in all later tables.

Table 14 — Threats of the training shop

No.	Element	Type	Scenario
TH1	Web application (process)	S	An attacker guesses a customer’s password on the login page
TH2	Database (store)	T, I	SQL injection through unchecked input in the search form
TH3	Web application (process)	I, E	A customer puts in someone else’s order number and sees the data
TH4	Web application (process)	D	A flood of requests to the login page stops the site
TH5	Administrator (entity)	R	The administrator denies changing a product price
TH6	Payment request (flow)	T	An attacker changes the amount in a request to the payment service

5.4 Risk assessment

Sl. 25

There are usually more threats than we can fix. So we rank them by **risk**. ISO/IEC 27000 says risk is the effect of uncertainty on goals [3]. It is often shown as impact times likelihood. The simple way is to give each a score from 1 to 3 and multiply them (table 15). A score of 1–2 is low, 3–4 is medium and 6–9 is high. The scores here are only examples. In real work the team sets them from facts about the system.

Table 15 — Risk ranking of the shop threats

No.	Threat	Likelihood	Impact	Risk	Level
TH1	Password guessing	3	2	6	High
TH2	SQL injection	2	3	6	High
TH3	Access to others’ orders	3	3	9	High
TH4	Denial of service	2	2	4	Medium
TH5	Denying a price change	1	2	2	Low
TH6	Payment amount tampering	1	3	3	Medium

To rate single vulnerabilities we also use CVSS [14].

6 Zero trust architecture

6.1 Why zero trust

Sl. 27

The old model said: inside the network is trusted, outside is not. This fails when resources are in the cloud, people work remotely and attacks come through the supply chain, as in SolarWinds. An attacker who is inside can move freely. According to NIST SP 800-207 [15], **zero trust** is a set of ideas that help us make exact access decisions for every request, with least privilege, when we assume that the network is already compromised. Being inside the network gives no trust. Every access is checked. Table 16 shows the main differences.

Table 16 — Perimeter model and zero trust

Aspect	Perimeter model	Zero trust
Trust	Inside the network is trusted	Nobody is trusted by default
Checks	At network entry (VPN, domain)	Every access to a resource
Segmentation	Networks and firewalls	Fine-grained, based on identity
Movement inside	Free	Every hop is checked
Weakness	One breach opens everything	Hard to deploy

6.2 Tenets of zero trust

Sl. 28

NIST SP 800-207 gives seven tenets (table 17).

Table 17 — Zero trust tenets from NIST SP 800-207

No.	Tenet	Meaning
1	All data sources and computing services are resources	No special systems outside protection
2	All communication is secured, wherever the network is	The internal network is not safe
3	Access to a resource is given per session	No permanent trust
4	Access follows a dynamic policy: identity, device state, behavior	The context of the request counts
5	The organization watches the state of its devices	Do not trust a weak device
6	Authentication and authorization happen before access is given	Check before access, not after
7	The organization collects data about resources and traffic to improve protection	Logs improve policies

6.3 Components

Sl. 29

Three logical parts make the access decision (figure 5). The **policy engine** decides whether to give access. The **policy administrator** opens or closes the session following this decision. The **policy enforcement point** sits on the traffic path. It lets the request pass or blocks it. A subject never reaches a resource directly. All requests go through the enforcement point.

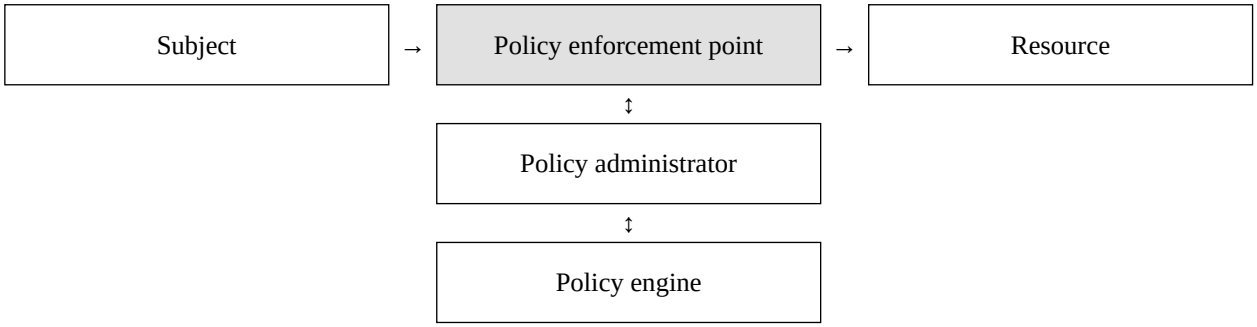


Figure 5 — Logical parts of zero trust

6.4 Zero trust in the shop

Sl. 30

In the shop, zero trust means this. The requests of an administrator are checked as strictly as those of a customer. Every access to an order checks the owner (TH3). Administrator actions need a second factor and are logged (TH5). The application gets only the database rights it needs, so an injection does less harm (TH2). The payment service accepts only signed requests and checks the amount (TH6).

7 Security requirements

7.1 Sources of requirements

Sl. 32

Requirements do not come from nowhere. They come from four sources (table 18).

Table 18 — Sources of security requirements

Source	What it gives	Example
Business and contracts	Promises to customers and partners	Payment data protection, service level agreements
Industry standards	Tested lists of controls	OWASP ASVS, ISO 27001, NIST CSF, CIS Controls
Threat modeling	Controls against the threats of this system	For each STRIDE threat: a control, a test, a metric
Internal policies	Rules of the organization	99.9 % availability, retention periods, response rules

7.2 OWASP ASVS

Sl. 33

OWASP ASVS is a standard to verify the security of web applications [16]. Version 5.0 (2025) has about 350 requirements in 17 chapters and three levels. Level 1 fits any application. Level 2 is for applications with sensitive data. Level 3 is for critical systems such as healthcare and finance. The levels add up: level 3 includes levels 1 and 2.

7.3 How to write a requirement

Sl. 34

A good requirement is **specific**, **verifiable**, **traceable** and **measurable** (table 19).

Table 19 — Properties of a security requirement

Property	What is needed	Example
Specific	What exactly to build	Lock the account after 5 failed logins in 15 minutes
Verifiable	How to confirm it works	Test: six requests with a wrong password, the sixth gets HTTP 429
Traceable	Which threat or standard it comes from	Password guessing (STRIDE: spoofing)
Measurable	Which metric shows it works in production	Number of lockouts per day

A full entry for one threat has six parts: the threat (TH1: password guessing); the requirement (lockout after five failures in 15 minutes); the implementation (a counter of failures); the test (an automatic test with six requests); the log event (`auth.lockout` with user, address, time and number of attempts); and the metric (number of lockouts, with an alert above a limit).

7.4 Threat modeling process

Sl. 35

Threat modeling is a form of risk assessment. It models the attack and the defense of an object such as data, an application or a system [6]. In practice it joins all earlier steps into nine steps (figure 6).

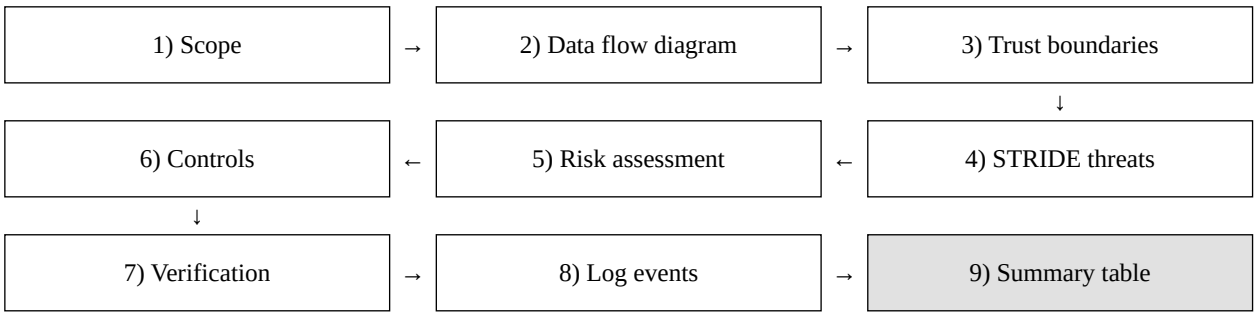


Figure 6 — Nine steps of threat modeling

Sl. 36

The result is a table that links each threat to a control, a test and a log event (table 20). Developers use it as a list of requirements. The monitoring team uses it as a list of events to detect.

Table 20 — Summary table “threat — control — verification — event” for the shop

Threat	Type	Risk	Control	Verification	Event
TH1. Password guessing	S	High	Second factor, rate limit, lockout	Six wrong attempts → lockout	`auth.lockout`
TH2. SQL injection	T, I	High	Parameterized queries, web firewall	Static analysis, sqlmap	`waf.block.sqli`
TH3. Others’ orders	I, E	High	Order owner check	Another’s order → 403	`authz.denied`
TH4. Denial of service	D	Medium	Rate limit, CDN	Load test	`rate.exceeded`
TH5. Price change denied	R	Low	Tamper-evident log, second factor	Price change → entry with author	`admin.price_change`
TH6. Payment amount	T	Medium	Request signing, amount check	Changed amount → rejected	`payment.mismatch`

8 Conclusion

Sl. 37

Table 21 lists the main points of the lecture.

Table 21 — Main points of the lecture

Chapter	Main point
1. Properties and principles	Confidentiality, integrity, availability; eight principles; defense in depth
2. Assets and surface	Protect assets; reduce and review the attack surface
3. Trust boundaries	Check where trust changes; the DFD shows the boundaries
4. Adversary and threats	Adversary model, attack chain, STRIDE, risk ranking
5. Zero trust	Every access is checked by the engine, the administrator and the enforcement point
6. Requirements	Specific, verifiable, traceable, measurable; result is the threat table

In practice 1 you do the full cycle for a training system: a data flow diagram, STRIDE threats, risk ranking, the table “threat — control — verification — event” and a list of requirements.

References

[1] R. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed. Wiley, 2020.

[2] Federal Trade Commission, “Equifax Data Breach Settlement.” [Online]. Available: https://www.ftc.gov/enforcement/refunds/equifax-data-breach-settlement

[3] ISO/IEC, “ISO/IEC 27000:2018. Information technology — Security techniques — Information security management systems — Overview and vocabulary.” 2018.

[4] J. H. Saltzer and M. D. Schroeder, “The Protection of Information in Computer Systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[5] National Institute of Standards and Technology, “Engineering Trustworthy Secure Systems,” technical report SP 800-160 Vol. 1 Rev. 1, 2022. doi: 10.6028/NIST.SP.800-160v1r1.

[6] National Institute of Standards and Technology, “Security and Privacy Controls for Information Systems and Organizations,” technical report SP 800-53 Rev. 5, 2020. doi: 10.6028/NIST.SP.800-53r5.

[7] U.S. Department of Justice, “Seattle Tech Worker Arrested for Data Theft Involving Large Financial Services Company.” [Online]. Available: https://www.justice.gov/usao-wdwa/pr/seattle-tech-worker-arrested-data-theft-involving-large-financial-services-company

[8] CISA, “Alert AA20-352A: Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations.” [Online]. Available: https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a

[9] Microsoft, “Data-flow diagram elements. Microsoft Learn: Threat modeling.” [Online]. Available: https://learn.microsoft.com/en-us/training/modules/tm-create-a-threat-model-using-foundational-data-flow-diagram-elements/1b-elements

[10] National Institute of Standards and Technology, “Guide for Conducting Risk Assessments,” technical report SP 800-30 Rev. 1, 2012. doi: 10.6028/NIST.SP.800-30r1.

[11] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, “Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains,” technical report, 2011.

[12] MITRE, “MITRE ATT&CK.” [Online]. Available: https://attack.mitre.org/

[13] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014.

[14] FIRST, “Common Vulnerability Scoring System, version 4.0: Specification Document.” [Online]. Available: https://www.first.org/cvss/v4-0/

[15] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” technical report SP 800–207, 2020. doi: 10.6028/NIST.SP.800-207.

[16] OWASP Foundation, “Application Security Verification Standard (ASVS), version 5.0.” [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

9 Self-check questions

1. Name five of the eight design principles and give an example of breaking each.

Answer. Least privilege (Capital One: a role with too much access). Separation of privilege (SolarWinds: a backdoor entered the build without independent review). Economy of mechanism (Log4Shell: an extra lookup function in a logging library). Fail-safe defaults (if the permission check fails, reject the request). Complete mediation (check the token on every request, not only at login).

2. How is a trust boundary different from a network perimeter?

Answer. A perimeter separates “inside” from “outside” and trusts the inside. A trust boundary exists wherever the level of trust changes, also inside the perimeter. Examples are service to database and user to administrator. So a system has more boundaries than perimeters.

3. Why do we need an adversary model?

Answer. It tells us who we defend against, with what means. Threats from likely adversaries get high priority. Attacks that these adversaries cannot do need no spending.

4. What is the difference between STRIDE and risk assessment?

Answer. STRIDE sorts threats and helps to find them (step 4). Risk assessment says which threats matter more (step 5).

5. Name the three logical parts of zero trust and what each does.

Answer. The policy engine decides on access. The policy administrator opens or closes the session. The policy enforcement point lets traffic pass or blocks it.

6. How do you make the requirement “protect against password guessing” verifiable?

Answer. Give a number and a time (“lockout after 5 failures in 15 minutes”). Give a test (six wrong requests, the sixth gets HTTP 429). Name the log event (`auth.lockout`) and the metric (lockouts per day).